

# COURSE TERM PROJECT

Week 15 Checkpoint

---

**Team:**水色真珠音調~用閃亮的歌聲開始現場演唱

組長/組員：廖祥恩

---

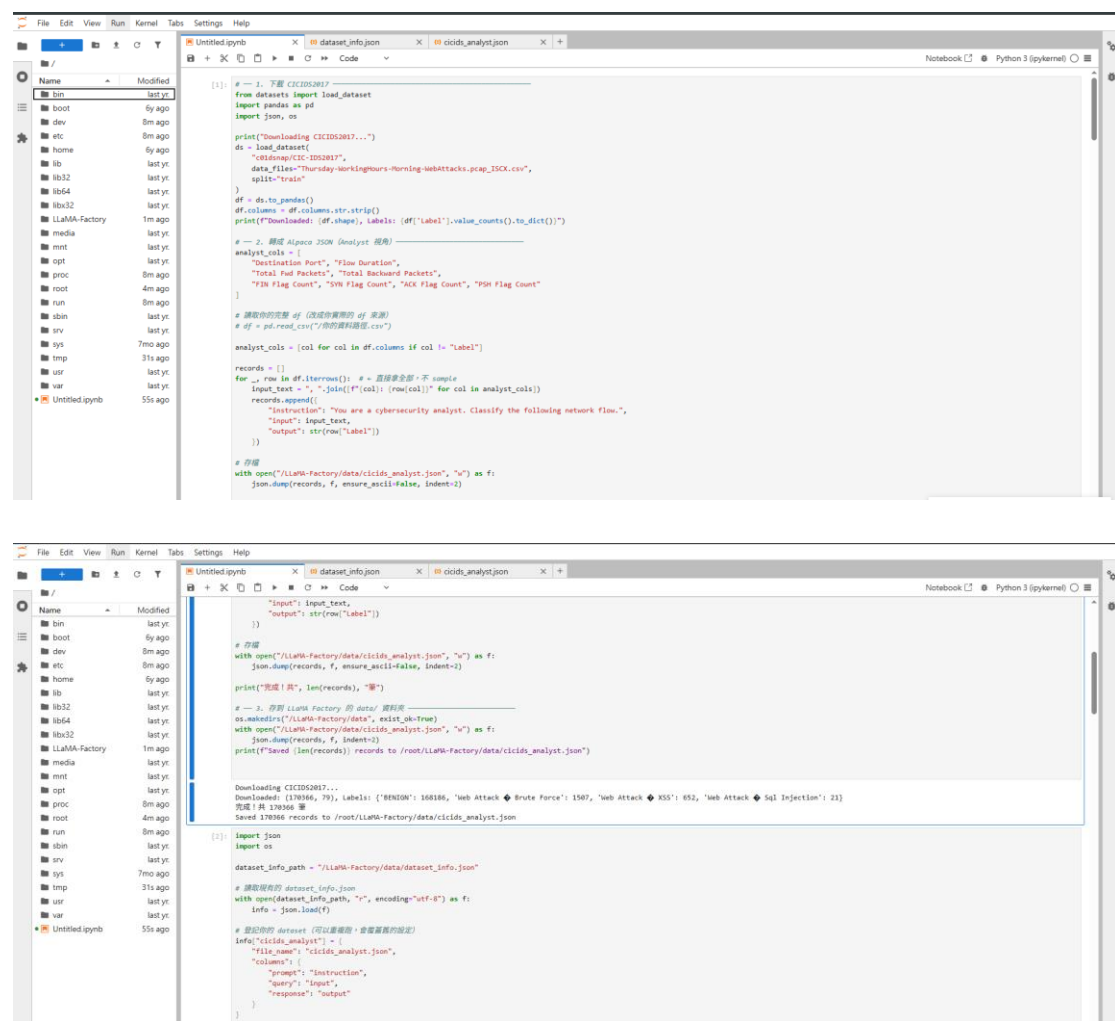
# 本周目標

實作 baseline model 並測試，透過 chat 和 test & evaluation 觀察其表現。

而我選定的 model 從原先預定的 model 改成 Llama-3.1-8B-Chinese-Chat。

## Baseline Finetunning (training1 & 2)

### Import dataset



```
[1]: # -- 1. 下載 CICIDS2017
from datasets import load_dataset
import pandas as pd
import json, os

print("Downloading CICIDS2017...")
ds = load_dataset(
    "cidsnap/CIC-IDS2017",
    data_files="Thursday-Northinghams-Morning-webattacks.pcap_35CX.csv",
    split="train"
)

df = ds.to_pandas()
df.columns = df.columns.str.strip()
print(f"Downloaded: {df.shape}, Labels: {df['Label'].value_counts().to_dict()}")

# -- 2. 讀取 Alpacas JSON (Analyst 視角)
analyst_cols = [
    "Destination Port", "Flow Duration",
    "Total Fwd Packets", "Total Backward Packets",
    "FIN Flag Count", "SYN Flag Count", "ACK Flag Count", "PSH Flag Count"
]

# 讀取你的花籃 df (改成你實際的 df 來源)
# df = pd.read_csv("/你的資料路徑.csv")

analyst_cols = [col for col in df.columns if col != "Label"]

records = []
for _, row in df.iterrows(): # = 直接拿全部，不 sample
    input_text = ", ".join([f"{col}: {row[col]}" for col in analyst_cols])
    records.append({
        "instruction": "You are a cybersecurity analyst. Classify the following network flow.",
        "input": input_text,
        "output": str(row["Label"])
    })

# 存檔
with open("/Llama-Factory/data/cicids_analyst.json", "w") as f:
    json.dump(records, f, ensure_ascii=False, indent=2)

# -- 3. 存到 Llama Factory 的 data/ 資料夾
os.makedirs("/Llama-Factory/data", exist_ok=True)
with open("/Llama-Factory/data/cicids_analyst.json", "w") as f:
    json.dump(records, f, indent=2)
print(f"Saved {len(records)} records to /root/.Llama-Factory/data/cicids_analyst.json")

Downloaded CICIDS2017...
Downloaded: (178946, 79), Labels: {'BENIGN': 168886, 'Web Attack - Brute Force': 1507, 'Web Attack - XSS': 852, 'Web Attack - Sql Injection': 23}
完成！共 178966 筆
Saved 178966 records to /root/.Llama-Factory/data/cicids_analyst.json

[2]: import json
import os

dataset_info_path = "/Llama-Factory/data/dataset_info.json"

# 讀取現有的 dataset_info.json
with open(dataset_info_path, "r", encoding="utf-8") as f:
    info = json.load(f)

# 添加你的 dataset (可以重複，重複讀取的話)
info["cicids_analyst"] = [
    {
        "file_name": "cicids_analyst.json",
        "columns": [
            "prompt": "Instruction",
            "query": "Input",
            "response": "Output"
        ]
    }
]
```

```
[2]: import json
import os

dataset_info_path = "/LLaMA-Factory/data/dataset_info.json"

# 讀取現有的 dataset_info.json
with open(dataset_info_path, "r", encoding="utf-8") as f:
    info = json.load(f)

# 登記你的 dataset (可以重複跑，會覆蓋舊的設定)
info["cicids_analyst"] = {
    "file_name": "cicids_analyst.json",
    "columns": {
        "prompt": "instruction",
        "query": "input",
        "response": "output"
    }
}

# 寫回去
with open(dataset_info_path, "w", encoding="utf-8") as f:
    json.dump(info, f, ensure_ascii=False, indent=2)

print("✅ 登記成功!")
print("目前所有 datasets:")
for key in info.keys():
    print(f" - {key}")

# 確認 json 檔案存在
json_path = "/LLaMA-Factory/data/cicids_analyst.json"
if os.path.exists(json_path):
    with open(json_path) as f:
        data = json.load(f)
    print(f"\n✅ cicids_analyst.json 存在，共 {len(data)} 筆資料")
else:
    print("\n❌ cicids_analyst.json 不存在！請先執行資料轉換的 code")
```

```
✅ 登記成功！
目前所有 datasets:
- identity
- alpaca_en_demo
- alpaca_zh_demo
- glaive_toolcall_en_demo
- glaive_toolcall_zh_demo
- mllm_demo
- mllm_audio_demo
- mllm_video_demo
- mllm_video_audio_demo
- alpaca_en
- alpaca_zh
- alpaca_gpt4_en
- alpaca_gpt4_zh
- glaive_toolcall_en
- glaive_toolcall_zh
- lima
- guanaco
- belle_2m
- belle_1m
- belle_0.5m
- belle_dialog
- belle_math
- belle_multiturn
- ultra_chat
- open_platypus
- codealpaca
- alpaca_cot
- openorca
- slimorca
- mathinstruct
- firefly
- wikiqa
- webqa
- webnovel
- nectar_sft
- deepctrl
- adgen_train
- adgen_eval
- sharegpt_hyper
- sharegpt4
- ultrachat_200k
- agent_instruct
- lmsys_chat
```

```
- agent_instruct
- lmsys_chat
- evol_instruct
- glaive_toolcall_100k
- cosmopedia
- steve_zh
- ruozhiba_gpt4
- neo_sft
- magpie_pro_300k
- magpie_ultra
- web_instruct
- openol_sft
- open_thoughts
- open_rl_math
- chinese_rl_distill
- llava_1k_en
- llava_1k_zh
- llava_150k_en
- llava_150k_zh
- pokemon_cap
- mllm_pt_demo
- oasst_de
- dolly_15k_de
- alpaca_gpt4_de
- openchabellier_de
- evol_instruct_de
- dolphin_de
- booksum_de
- airoboros_de
- ultrachat_de
- dpo_en_demo
- dpo_zh_demo
- dpo_mix_en
- dpo_mix_zh
- ultrafeedback
- coig_p
- rlhf_v
- vfeedback
- orca_pairs
- hh_rlhf_en
- nectar_rm
- orca_dpo_de
- kto_en_demo
- kto_mix_en
- ultrafeedback_kto
- wiki_demo
- c4_demo
- refinedweb
- redpajama_v2
```

```
- orca_dpo_de
- kto_en_demo
- kto_mix_en
- ultrafeedback_kto
- wiki_demo
- c4_demo
- refinedweb
- redpajama_v2
- wikipedia_en
- wikipedia_zh
- pile
- skypile
- fineweb
- fineweb_edu
- the_stack
- starcoder_python
- cicids_analyst
```

✅ cicids\_analyst.json 存在，共 170366 筆資料

## Model

Llama-3.1-8B-Chinese-Chat

# Training 1

## Parameter

Language  
en

Model name  
Input the initial name to search for the model.  
Llama-3-8B-Chinese-Chat

Model path  
Path to pretrained model or model identifier from Hugging Face.  
shenzhi-wang/Llama3-8B-Chinese-Chat

Finetuning method  
lora

Checkpoint path

Quantization bit  
Enable quantization (QLoRA).  
none

Quantization method  
Quantization algorithm to use.  
bnb

Chat template  
The chat template used in constructing prompts.  
llama3

RoPE scaling  
RoPE scaling method to use.  
none

Booster  
Approach used to boost training speed.  
auto

Train

Evaluate & Predict

Chat

Export

Stage  
The stage to perform in training.  
Supervised Fine-Tuning

Data dir  
Path to the data directory.  
data

Dataset  
cicids\_analyst

Preview dataset

Learning rate  
Initial learning rate for AdamW.  
5e-5

Epochs  
Total number of training epochs to perform.  
5.0

Maximum gradient norm  
Norm for gradient clipping.  
1.0

Max samples  
Maximum samples per dataset.  
500

Compute type  
Whether to use mixed precision training.  
bf16

Cutoff length  
Max tokens in input sequence.  
512

Batch size  
Number of samples processed on each GPU.  
4

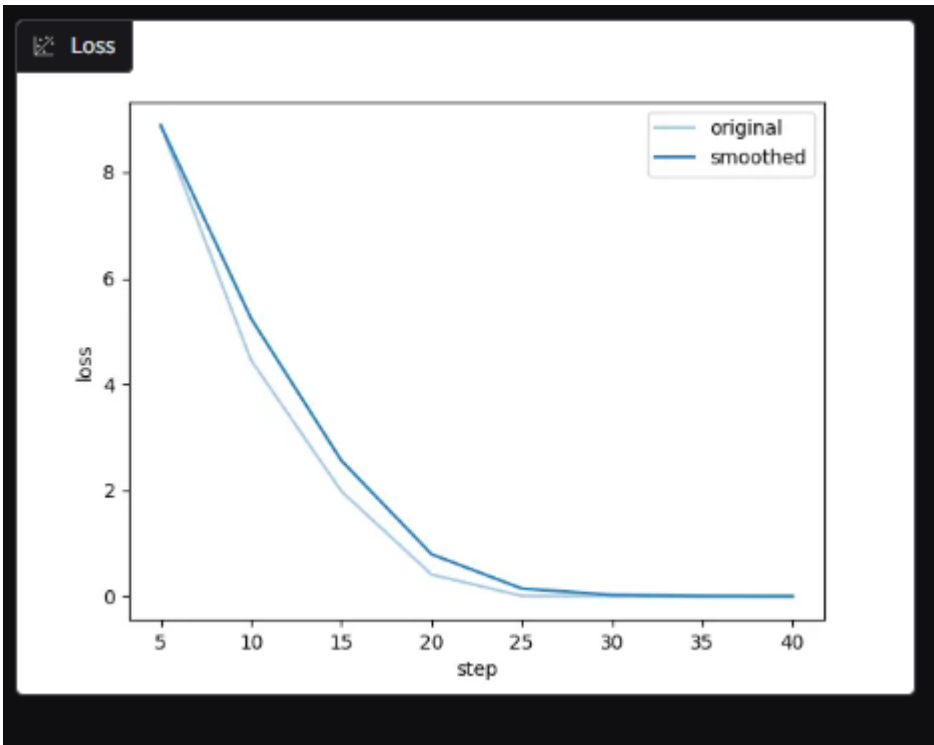
Gradient accumulation  
Number of steps for gradient accumulation.  
4

Val size  
Percentage of validation set from the entire dataset.  
0.1

LR scheduler  
Name of the learning rate scheduler.  
cosine

## Result

### Loss diagram



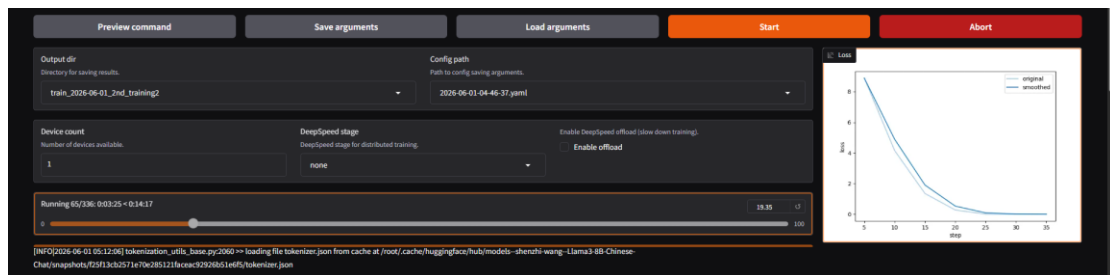
# Training 2

## Parameter

The screenshot shows the Llama Studio training interface with the following parameters:

- Stage:** Supervised Fine-Tuning
- Data dir:** data
- Dataset:** ckddu\_analyse
- Learning rate:** 5e-5
- Epochs:** 2.0
- Maximum gradient norm:** 1.0
- Max samples:** 3000
- Compute type:** bf16
- Cutoff length:** 131072
- Batch size:** 4
- Gradient accumulation:** 4
- Val size:** 0.1
- LR scheduler:** cosine

## Result



## Baseline Finetuning (training3)

由於前兩次透過修 max samples(500 -> 3000)無法解決過擬合的問題，後來我猜是因為資料量太，因此重新載入資料，重一個 label 變成多一個 label，資料量從 300 變成 5532 筆，並從取 max samples 為 3000 測試看可不可以改變 loss diagram 的表現。把剩餘 2000 筆左右的資料作為 test dataset 客觀測試模型微調後的表現。

# Import dataset

```
from datasets import load_dataset
import pandas as pd
import json, os

# -- 1. 讀入多個 CSV --
files = [
    "Thursday-workingHours-Morning-WebAttacks.pcap_ISCX.csv", # Web Attacks
    "Tuesday-workingHours.pcap_ISCX.csv", # FTP/SSH Brute Force
    "Wednesday-workingHours.pcap_ISCX.csv", # DoS / Heartbleed
    "Friday-workingHours-Afternoon-DoS.pcap_ISCX.csv", # DoS
    "Friday-workingHours-Afternoon-PortScan.pcap_ISCX.csv", # PortScan
]

dfs = []
for f in files:
    print(f"Downloading {f}...")
    ds = load_dataset('citronsp/CIC-IDS2017', data_files=f, split='train')
    df_tmp = ds.to_pandas()
    df_tmp.columns = df_tmp.columns.str.strip()
    print(f"  讀入數據量: {df_tmp.shape}")
    df_tmp['label'] = df_tmp['label'].value_counts().to_dict()
    dfs.append(df_tmp)
except Exception as e:
    print(f"❌ 讀入失敗: {e}")

df = pd.concat(dfs, ignore_index=True)
print(f"讀入數據總量: {df.shape}")
print(df['label'].value_counts())

# -- 2. 定義 Label 完整說明 --
label_description = {
    "BENIGN": "This network flow is benign. No malicious activity detected.",
    "Web Attack \u2013 Brute Force": "This network flow is a Web Attack \u2013 Brute Force. The attacker is repeatedly attempting login credentials.",
    "Web Attack \u2013 XSS": "This network flow is a Web Attack \u2013 XSS (Cross-Site Scripting). Malicious scripts are being injected into web requests.",
    "Web Attack \u2013 SQL Injection": "This network flow is a Web Attack \u2013 SQL Injection. Malicious SQL commands are embedded in the request to manipulate the database.",
    "FTP-Patator": "This network flow is an FTP Brute Force attack (Patator). Repeated FTP login attempts are made to guess valid credentials.",
    "SSH-Patator": "This network flow is an SSH Brute Force attack (Patator). Repeated SSH login attempts are made to gain unauthorized access.",
    "DoS-Hulk": "This network flow is a DoS Hulk attack. A high volume of HTTP requests overwhelms the web server.",
    "DoS-GoldenEye": "This network flow is a DoS GoldenEye attack. HTTP keep-alive connections are abused to exhaust server resources.",
    "DoS-Slowloris": "This network flow is a DoS Slowloris attack. Partial HTTP requests are sent slowly to hold connections open.",
    "DoS-Slowhttptest": "This network flow is a DoS Slowhttptest attack. Slow HTTP requests exhaust the web server connection pool.",
    "Heartbleed": "This network flow is a Heartbleed attack. The OpenSSL vulnerability is exploited to leak sensitive memory from the server.",
    "DoS": "This network flow is a DoS attack. Massive traffic from multiple sources overwhelms the target system.",
    "PortScan": "This network flow is a PortScan attack. The attacker is scanning ports to discover open services and vulnerabilities."
}

analyst_cols = [col for col in df.columns if col != "label"]

# -- 3. 平衡數據 (每個 Label 最多 500 筆) --
df_balanced = df.groupby("label", group_keys=False).apply(
    lambda x: x.sample(min(len(x), 500), random_state=42),
    include_groups=False
)
print(f"平衡後: {df_balanced['label'].value_counts()}")

# -- 3. 平衡數據 (每個 Label 最多 500 筆) --
df_balanced = (
    df.groupby("label", group_keys=False)
    .apply(lambda x: x.sample(min(len(x), 500), random_state=42))
    .reset_index(drop=True)
)
print(f"平衡後: {df_balanced['label'].value_counts()}")

# -- 4. 生成 Alpaca JSON --
records = []
for _, row in df_balanced.iterrows():
    label = str(row["label"]).strip()
    input_text = ", ".join([f"{col}: {row[col]}" for col in analyst_cols])
    output_text = label_description.get(label, f"This network flow is classified as {label}.")
    records.append({
        "instruction": "You are a cybersecurity analyst. Analyze the following network flow and classify it.",
        "input": input_text,
        "output": output_text
    })

# -- 5. 存檔 --
os.makedirs("LLaMA-Factory/data", exist_ok=True)
save_path = "LLaMA-Factory/data/cicids_analyst.json"
with open(save_path, "w") as f:
    json.dump(records, f, ensure_ascii=False, indent=2)

print(f"完成! 共 {len(records)} 筆")
print(f"儲存至: {save_path}")
print(f"Label 分布:")
from collections import Counter
print(Counter([r["output"][:140] for r in records]))

Downloading Thursday-workingHours-Morning-WebAttacks.pcap_ISCX.csv...
  (170366, 79), Labels: ('BENIGN': 168186, 'Web Attack - Brute Force': 1507, 'Web Attack - XSS': 652, 'Web Attack - SQL Injection': 21)
Downloading Tuesday-workingHours.pcap_ISCX.csv...
  (445909, 79), Labels: ('BENIGN': 432074, 'FTP-Patator': 7938, 'SSH-Patator': 5897)
Downloading Wednesday-workingHours.pcap_ISCX.csv...
  (692703, 79), Labels: ('BENIGN': 440031, 'DoS Hulk': 231073, 'DoS GoldenEye': 10293, 'DoS Slowloris': 5796, 'DoS Slowhttptest': 5499, 'Heartbleed': 11)
Downloading Friday-workingHours-Afternoon-DoS.pcap_ISCX.csv...
  (225745, 79), Labels: ('DoS': 128027, 'BENIGN': 97718)
Downloading Friday-workingHours-Afternoon-PortScan.pcap_ISCX.csv...
  (286467, 79), Labels: ('PortScan': 158930, 'BENIGN': 127537)
```

合併後總筆數: (1021190, 79)

| Label                                                                                                      |         |
|------------------------------------------------------------------------------------------------------------|---------|
| BENIGN                                                                                                     | 1265546 |
| DoS Hulk                                                                                                   | 231073  |
| PortScan                                                                                                   | 158930  |
| DDoS                                                                                                       | 128027  |
| DoS GoldenEye                                                                                              | 10293   |
| FTP-Patator                                                                                                | 7938    |
| SSH-Patator                                                                                                | 5897    |
| DoS slowloris                                                                                              | 5796    |
| DoS slowhttptest                                                                                           | 5499    |
| Web Attack  Brute Force   | 1507    |
| Web Attack  XSS           | 652     |
| Web Attack  Sql Injection | 21      |
| Heartbleed                                                                                                 | 11      |

Name: count, dtype: int64

/tmp/ipykernel\_557/2476505701.py:59: DeprecationWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. (Either pass 'include\_groups=False' to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.)  
.apply(lambda x: x.sample(min(len(x), 500), random\_state=42))

平衡後:

| Label                                                                                                      |     |
|------------------------------------------------------------------------------------------------------------|-----|
| BENIGN                                                                                                     | 500 |
| DDoS                                                                                                       | 500 |
| DoS GoldenEye                                                                                              | 500 |
| DoS Hulk                                                                                                   | 500 |
| DoS slowhttptest                                                                                           | 500 |
| DoS slowloris                                                                                              | 500 |
| FTP-Patator                                                                                                | 500 |
| PortScan                                                                                                   | 500 |
| SSH-Patator                                                                                                | 500 |
| Web Attack  XSS           | 500 |
| Web Attack  Brute Force   | 500 |
| Web Attack  Sql Injection | 21  |
| Heartbleed                                                                                                 | 11  |

Name: count, dtype: int64

 完成！共 5532 筆

儲存至: /LLaVA-Factory/data/cicids\_analyst.json

Label 分布:

Counter({'This network flow is classified as Web A': 3021, 'This network flow is benign. No malicious': 500, 'This network flow is a DDoS attack. Pass': 500, 'This network flow is a DoS GoldenEye att': 500, 'This network flow is a DoS Hulk attack. ': 500, 'This network flow is a DoS SlowHTTPTest ': 500, 'This network flow is a DoS Slowloris att': 500, 'This network flow is an FTP Brute Force ': 500, 'This network flow is a PortScan attack. ': 500, 'This network flow is an SSH Brute Force ': 500, 'This network flow is a Heartbleed attack': 11})

[7]: import json

dataset\_info\_path = "/LLaVA-Factory/data/dataset\_info.json"

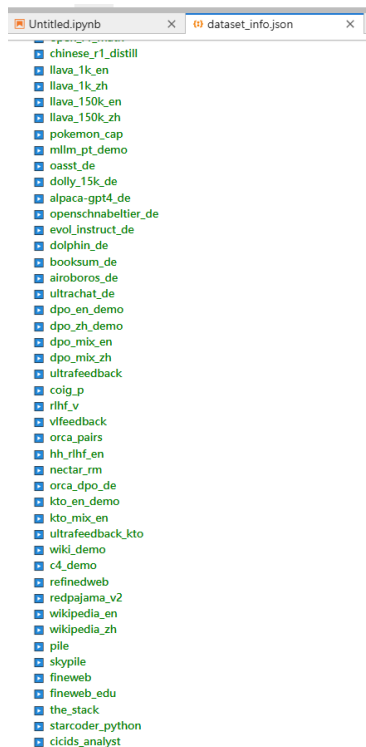
with open(dataset\_info\_path, "r", encoding="utf-8") as f:  
info = json.load(f)

info["cicids\_analyst"] = {  
"file\_name": "cicids\_analyst.json",  
"columns": {  
"prompt": "Instruction",  
"query": "Input",  
"response": "Output"  
}  
}

with open(dataset\_info\_path, "w", encoding="utf-8") as f:  
json.dump(info, f, ensure\_ascii=False, indent=2)

print( 登記成功！)

 登記成功！



# Parameter

Language

en

Model name

Input the initial name to search for the model.

Llama-3.1-8B-Chinese-Chat

Model path

Path to pretrained model or model identifier from Hugging Face.

shenN-wang/Llama3.1-8B-Chinese-Chat

Finetuning method

lora

Checkpoint path

Quantization bit

Enable quantization (QnB).

none

Quantization method

Quantization algorithm to use.

bnb

Chat template

The chat template used in constructing prompts.

llama3

RoPE scaling

RoPE scaling method to use.

none

Booster

Approach used to boost training speed.

auto

Train

Evaluate & Predict

Chat

Export

Stage

The stage to perform in training.

Supervised Fine-Tuning

Data dir

Path to the data directory.

data

Dataset

cicids\_analyst

Preview dataset

Learning rate

Initial learning rate for AdamW.

5e-5

Epochs

Total number of training epochs to perform.

3.0

Maximum gradient norm

Norm for gradient clipping.

1.0

Max samples

Maximum samples per dataset.

3000

Compute type

Whether to use mixed precision training.

bfloat16

Cutoff length

Max tokens in input sequence.

512

Batch size

Number of samples processed on each GPU.

2

Gradient accumulation

Number of steps for gradient accumulation.

4

Val size

Percentage of validation set from the entire dataset.

0.1

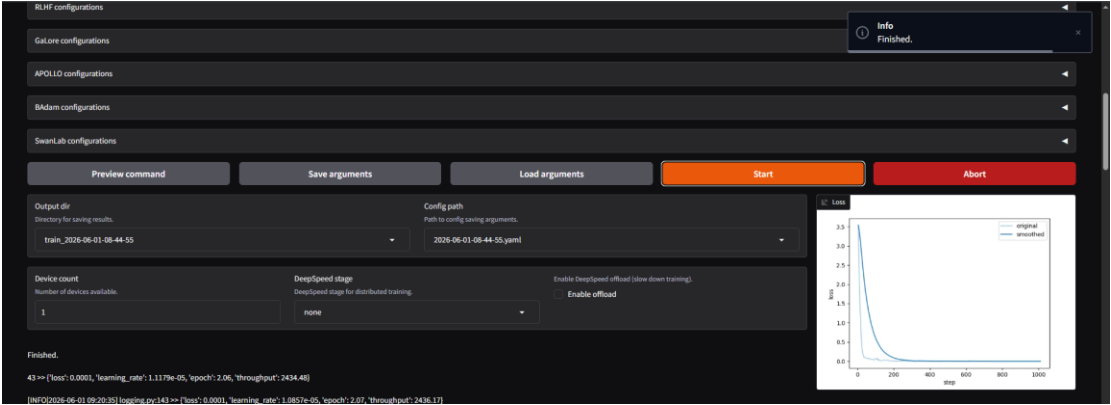
LR scheduler

Name of the learning rate scheduler.

cosine

# Result

# Loss diagram



# Chat test

# Test sentence

正常流量測試 測試 1 — BENIGN

Analyze the following network traffic and classify it: Flow Duration: 119261000, Total Fwd Packets: 20, Total Backward Packets: 18, Flow Bytes/s: 1024.5, Flow Packets/s: 45.2, SYN Flag Count: 1, ACK Flag Count: 35

攻擊流量測試 測試 2 — DDoS

Analyze the following network traffic and classify it: Flow Duration: 2341, Total Fwd Packets: 1500, Total Backward Packets: 0, Flow Bytes/s: 985432.1, Flow

Packets/s: 6408.4, SYN Flag Count: 1500, ACK Flag Count: 0

### 測試 3 — PortScan

Analyze the following network traffic and classify it: Flow Duration: 51000, Total Fwd Packets: 1, Total Backward Packets: 0, Flow Bytes/s: 78.4, Flow Packets/s: 19.6, SYN Flag Count: 1, RST Flag Count: 1, Destination Port: 445

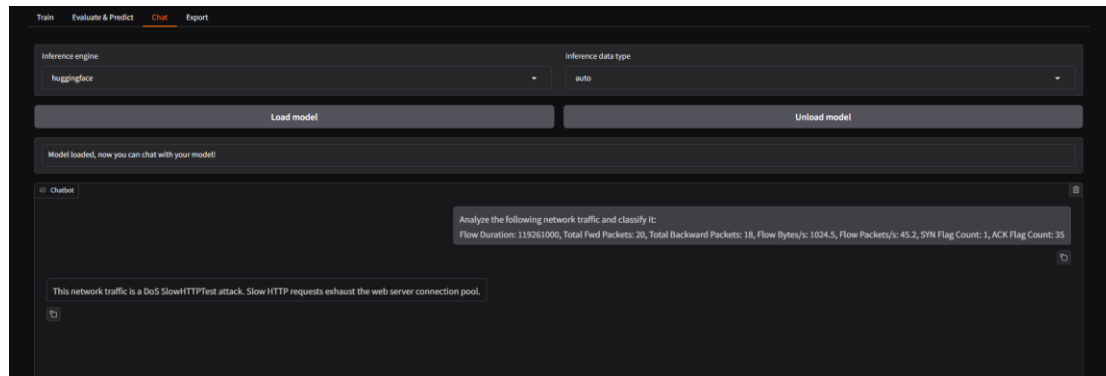
### 測試 4 — Brute Force (FTP)

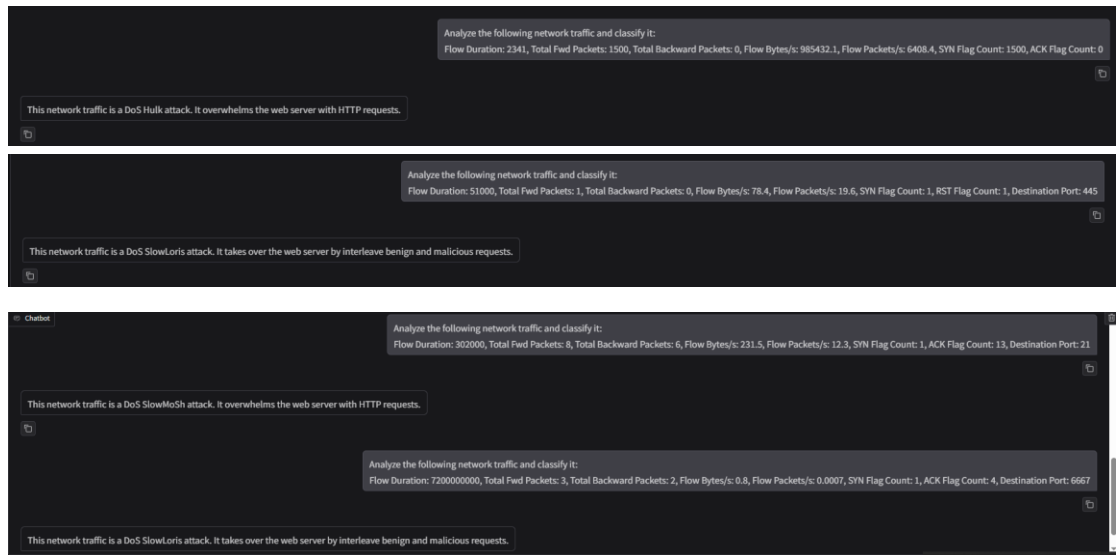
Analyze the following network traffic and classify it: Flow Duration: 302000, Total Fwd Packets: 8, Total Backward Packets: 6, Flow Bytes/s: 231.5, Flow Packets/s: 12.3, SYN Flag Count: 1, ACK Flag Count: 13, Destination Port: 21

### 測試 5 — Bot

Analyze the following network traffic and classify it: Flow Duration: 7200000000, Total Fwd Packets: 3, Total Backward Packets: 2, Flow Bytes/s: 0.8, Flow Packets/s: 0.0007, SYN Flag Count: 1, ACK Flag Count: 4, Destination Port: 6667

## Result





| 測試   | 正確標籤              | 模型預測             | 結果               |
|------|-------------------|------------------|------------------|
| 測試 1 | BENIGN            | DoS SlowHTTPTest | ✗ False Positive |
| 測試 2 | DDoS              | DoS Hulk         | ✓ 大方向正確          |
| 測試 3 | PortScan          | DoS SlowLoris    | ✗ 完全錯誤           |
| 測試 4 | Brute Force (FTP) | DoS SlowMoSh     | ✗ 完全錯誤           |
| 測試 5 | Bot               | DoS SlowLoris    | ✗ 完全錯誤           |

# Split Test Datasets

```
[9]: import json
import random
import os

# -- 1. 讀取全部資料 -----
with open("/LLaVA-Factory/data/cicids_analyst.json", "r") as f:
    records = json.load(f)

print(f"總筆數: {len(records)}")

# -- 2. 打亂並切分 3000 train / 剩下 test -----
random.seed(42)
random.shuffle(records)

train_data = records[:3000]
test_data = records[3000:]

print(f"Train: {len(train_data)} 筆")
print(f"Test: {len(test_data)} 筆")

# -- 3. 存 train (覆蓋原本的) -----
with open("/LLaVA-Factory/data/cicids_analyst.json", "w") as f:
    json.dump(train_data, f, ensure_ascii=False, indent=2)

# -- 4. 存 test -----
with open("/LLaVA-Factory/data/cicids_analyst_test.json", "w") as f:
    json.dump(test_data, f, ensure_ascii=False, indent=2)

# -- 5. 寫入 test 到 dataset_info.json -----
info_path = "/LLaVA-Factory/data/dataset_info.json"
with open(info_path, "r") as f:
    info = json.load(f)

info["cicids_analyst_test"] = {
    "file_name": "cicids_analyst_test.json",
    "columns": {
        "prompt": "instruction",
        "query": "input",
        "response": "output"
    }
}
```

```
# -- 3. 存 train (覆蓋原本的) -----
with open("/LLaVA-Factory/data/cicids_analyst.json", "w") as f:
    json.dump(train_data, f, ensure_ascii=False, indent=2)

# -- 4. 存 test -----
with open("/LLaVA-Factory/data/cicids_analyst_test.json", "w") as f:
    json.dump(test_data, f, ensure_ascii=False, indent=2)

# -- 5. 寫入 test 到 dataset_info.json -----
info_path = "/LLaVA-Factory/data/dataset_info.json"
with open(info_path, "r") as f:
    info = json.load(f)

info["cicids_analyst_test"] = {
    "file_name": "cicids_analyst_test.json",
    "columns": {
        "prompt": "instruction",
        "query": "input",
        "response": "output"
    }
}

with open(info_path, "w") as f:
    json.dump(info, f, ensure_ascii=False, indent=2)

print("\n✅ 完成!")
print(f"- 訓練資料: cicids_analyst.json ({len(train_data)} 筆)")
print(f"- 測試資料: cicids_analyst_test.json ({len(test_data)} 筆)")
print(f"- dataset_info.json 已更新")

總筆數: 5532
Train: 3000 筆
Test: 2532 筆

✅ 完成!
- 訓練資料: cicids_analyst.json (3000 筆)
- 測試資料: cicids_analyst_test.json (2532 筆)
- dataset_info.json 已更新
```

# Evaluate & test

## parameters

Language

Model name

Model path

en

Llama-3.1-8B-Chinese-Chat

shenzhi-wang/Llama3.1-8B-Chinese-Chat

Finetuning method

Checkpoint path

lora

train\_2026-06-01-08-44-55

Quantization bit

Quantization method

Chat template

RuPE scaling

Booster

none

bnb

Llama3

none

auto

Train

Evaluate & Predict

Chat

Export

Data dir

Dataset

Preview dataset

data

cicids\_analyst\_test

Cutoff length

Max samples

Batch size

Save predictions

1024

100000

2

1024

Maximum new tokens

Top-p

Temperature

Output dir

512

0.95

0.95

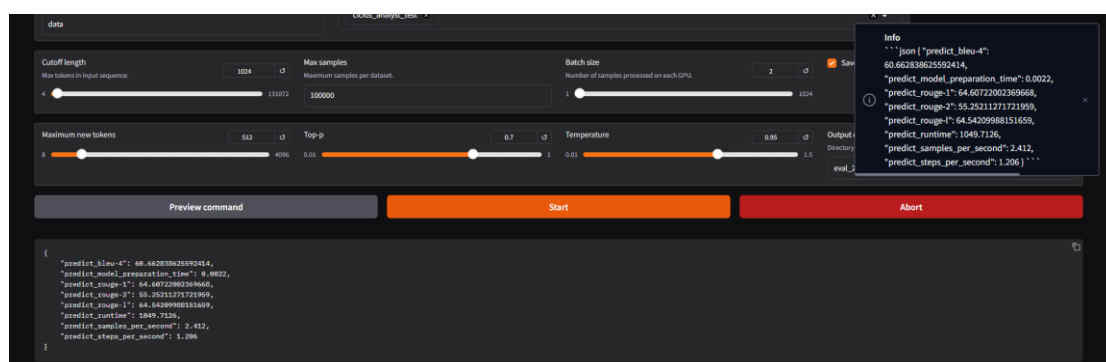
eval\_2026-06-01-08-44-55

Preview command

Start

Abort

## Score



| 指標                         | 數值      | 解讀 |
|----------------------------|---------|----|
| predict_bleu-4             | 60.66   | 優異 |
| predict_rouge-1            | 64.61   | 優異 |
| predict_rouge-2            | 55.25   | 良好 |
| predict_rouge-l            | 64.54   | 優異 |
| predict_runtime            | 1049.7s | 正常 |
| predict_samples_per_second | 2.41    | 正常 |

## Summary

透過上述結果觀察，我們可以發現量化指標優異，然而對於實際上的 Chat 的表現並不是很好，我推測有可能是基於資料量不平衡所造成的，雖然 datasets 有些不平衡導致實際表現不佳，然而若改變資料及改變的話，會導致實驗失真。因此，我將透仍使用相同資料集並觀察量化評分，如此一來較能表現出實驗組和 baseline model 的實際差距。